



DWN - Unid IV

Exercício 1:

Uma aplicação possui três necessidades de observabilidade:

1. registrar detalhes estruturados de uma falha de autenticação;
 2. contabilizar quantas falhas ocorreram ao longo do tempo;
 3. correlacionar as etapas executadas durante uma mesma operação.
- Qual conjunto associa corretamente cada necessidade ao mecanismo correspondente?

A)

Health checks, ILogger e output cache.

B)

ActivitySource, ILogger e Counter<long>.

C)

Counter<long>, ActivitySource e ILogger.

D)

ILogger, Counter<long> e ActivitySource.

E)

ILogger, health checks e Counter<long>.

Exercício 2:

Uma aplicação registra uma verificação de banco de dados:

```
builder.Services
```

```
.AddHealthChecks()
```

```
.AddCheck<DatabaseHealthCheck>("database");
```

e expõe:

```
app.MapHealthChecks(
```

```
    "/health/live",
```

```
    new HealthCheckOptions
```

```
{
```

```
    Predicate = _ => false
```

```
});
```

```
app.MapHealthChecks("/health/ready");
```

O processo continua executando, mas o banco de dados fica indisponível. Qual resultado é coerente com essa configuração?

A)

Ambos os endpoints falham porque toda verificação registrada é executada em qualquer MapHealthChecks().

B)

/health/live falha e /health/ready permanece saudável porque readiness não pode consultar dependências externas.

C)

/health/live pode continuar saudável porque exclui as verificações registradas, enquanto /health/ready pode indicar falha em razão da indisponibilidade do banco.

D)

Ambos permanecem saudáveis porque health checks não devem representar disponibilidade de banco.

E)

/health/ready somente pode falhar quando o próprio processo ASP.NET Core for encerrado.

Exercício 3:

Uma aplicação consulta um serviço externo. A política deve:

- repetir a chamada até três vezes diante de falhas transitórias;
 - após falhas repetidas, interromper temporariamente novas chamadas à dependência;
 - durante esse período, permitir que a aplicação adote uma resposta alternativa.
- Quais padrões correspondem aos dois primeiros comportamentos?

A)

Rate limiting e output caching.

B)

Duas políticas de retry com intervalos diferentes.

C)

Circuit breaker e autenticação.

D)

Output caching e readiness check.

E)

Retry e circuit breaker.

Exercício 4:

Uma aplicação possui:

GET /api/catalogo → catálogo público

GET /api/catalogo/{id} → detalhe público

POST /api/auth/login → autenticação

GET /api/perfil → dados personalizados do usuário

Qual combinação de output cache e rate limiting é mais coerente?

A)

Considerar output cache para respostas públicas adequadas do catálogo, aplicar limite mais restritivo ao login e não tratar o perfil autenticado como resposta de

cache compartilhado.

B)

Utilizar cache compartilhado em /api/perfil e /api/auth/login e dispensar limites no login.

C)

Armazenar em cache todos os GET, inclusive /api/perfil, porque o verbo HTTP é suficiente para determinar se o compartilhamento é seguro.

D)

Aplicar rate limiting somente aos endpoints públicos porque a validação da senha já impede abuso do login.

E)

Aplicar exatamente a mesma política de cache e de limite a todos os endpoints para manter comportamento uniforme.

Exercício 5:

Uma aplicação registra políticas:

```
builder.Services.AddRateLimiter(...);  
builder.Services.AddOutputCache(...);
```

e configura:

```
app.MapGet("/api/dados", ObterDados)  
    .RequireRateLimiting("public-api")  
    .CacheOutput("public-cache");
```

Entretanto, não existem as chamadas:

```
app.UseRateLimiter();  
app.UseOutputCache();
```

Qual análise está correta?

A)

As políticas funcionam normalmente porque os metadados do endpoint substituem os middlewares.

B)

Os serviços e as políticas foram registrados e associados ao endpoint, mas os respectivos middlewares ainda precisam integrar o pipeline.

C)

O output cache funciona pelo endpoint, mas o rate limiting exige middleware.

D)

O rate limiting funciona pelo endpoint, mas o output cache exige middleware.

E)

A aplicação necessariamente falha na compilação porque `RequireRateLimiting()` e `CacheOutput()` só existem depois das chamadas `Use...()`.

Exercício 6:

Uma aplicação possui três tipos de configuração:

- nome público da aplicação;
- URL de um serviço usada especificamente em Staging;
- chave secreta de acesso a uma API externa.

A equipe propõe:

`appsettings.json` → nome público

`appsettings.Staging.json` → URL de Staging

User Secrets → chave durante desenvolvimento

variável de ambiente → chave real em produção

Qual avaliação é a mais adequada?

A)

Todas as configurações deveriam ficar em `appsettings.json`, pois arquivos específicos de ambiente e variáveis servem apenas ao sistema operacional.

B)

A chave deveria ficar em `appsettings.Production.json`, pois um arquivo específico de produção constitui armazenamento seguro de segredos.

C)

A separação é apropriada: valores não sensíveis podem ser organizados por ambiente, enquanto segredos devem permanecer fora dos arquivos versionados.

D)

Cada ambiente exige nova compilação porque variáveis de ambiente não participam do sistema de configuração do ASP.NET Core.

E)

User Secrets é o mecanismo indicado para distribuir segredos às máquinas de produção.

Exercício 7:

O arquivo de configuração contém:

```
{  
  "Sistema": {  
    "ServicoExterno": {  
      "ApiKey": ""  
    }  
  }  
}
```

Qual nome de variável de ambiente representa essa chave hierárquica de forma compatível entre plataformas no sistema de configuração do ASP.NET Core?

A)

Sistema.ServicoExterno.ApiKey

B)

Sistema__ServicoExterno__ApiKey

C)

Sistema::ServicoExterno::ApiKey

D)

Sistema_ServicoExterno_ApiKey

E)

Sistema/ServicoExterno/ApiKey

Exercício 8:

Considere o Dockerfile:

```
FROM mcr.microsoft.com/dotnet/aspnet:10.0 AS base  
WORKDIR /app
```

```
FROM mcr.microsoft.com/dotnet/sdk:10.0 AS build  
WORKDIR /src
```

```
COPY ["Aplicacao.csproj", "."]
```

```
RUN dotnet restore "Aplicacao.csproj"
```

```
COPY . .
```

```
RUN dotnet publish "Aplicacao.csproj" \  
    -c Release -o /app/publish
```

```
FROM base AS final
```

```
WORKDIR /app
```

```
COPY --from=build /app/publish .
```

```
ENTRYPOINT ["dotnet", "Aplicacao.dll"]
```

Qual é uma finalidade importante dessa construção em múltiplos estágios?

A)

Manter o SDK e o código-fonte na imagem final para recompilar a aplicação sempre que o contêiner iniciar.

B)

Incorporar automaticamente as variáveis de ambiente de produção ao conteúdo permanente da imagem.

C)

Eliminar a etapa de publicação, pois a imagem aspnet executa diretamente arquivos .cs.

D)

Separar as ferramentas necessárias ao build das dependências necessárias à execução, copiando para a imagem final os artefatos publicados.

E)

Executar simultaneamente uma instância de desenvolvimento no estágio build e outra de produção no estágio final.

Exercício 9:

Uma solução full-stack pode utilizar uma destas estratégias:

I. O ASP.NET Core serve a API e os arquivos compilados do frontend.

II. Frontend e API são implantados separadamente e possuem ciclos de implantação independentes.

Qual comparação está correta?

A)

A estratégia I pode facilitar uma implantação conjunta e o uso da mesma origem. A estratégia II permite maior independência de implantação, mas exige coordenação entre os contratos utilizados pelo frontend e pela API.

B)

A estratégia I exige que frontend e API utilizem domínios diferentes. A estratégia II exige que ambos utilizem o mesmo domínio.

C)

A estratégia II elimina a necessidade de compatibilidade entre frontend e API porque cada aplicação possui seu próprio pipeline.

D)

A estratégia I impede que arquivos estáticos sejam armazenados em cache, enquanto a estratégia II permite cache apenas no frontend.

E)

Na estratégia II, alterações incompatíveis na API não afetam versões anteriores do frontend porque os dois sistemas são implantados separadamente.

Exercício 10:

Uma aplicação deve servir uma SPA apenas sob o prefixo /app, sem interferir nas rotas /Pets, /api/produtos, /health/live e /health/ready. A configuração é:

```
var indexPath = Path.Combine(  
    app.Environment.WebRootPath,  
    "app",  
    "index.html");
```

```
if (File.Exists(indexPath))  
{  
    app.MapFallbackToFile(  
        "/app/{*path:nonfile}",  
        "app/index.html");  
}
```

Qual é o efeito pretendido?

A)

Se index.html não existir, qualquer requisição para /app será redirecionada automaticamente para /Pets.

B)

File.Exists() verificará o arquivo apenas durante a compilação, de modo que o fallback será registrado mesmo quando a condição for falsa.

C)

O fallback fará /api/produtos devolver index.html sempre que uma consulta válida não encontrar registros.

D)

MapFallbackToFile() ignorará o template informado e capturará qualquer URL desconhecida da aplicação.

E)

Quando o arquivo existir, o fallback ficará restrito ao prefixo /app, permitindo que as rotas de MVC, API e health checks fora desse prefixo continuem sendo tratadas por seus endpoints.